

Rapport du Projet — Cryptographie Appliquée

Module : Cryptographie Appliquée
Niveau : Ingénierie 3 — Cybersécurité
Institution : USTHB (Université des Sciences et de la Technologie Houari Boumédiène)
Année universitaire : 2025–2026

Membres de l'équipe

Nom	Prénom
Bentafat	Wail
Chennit	Zineb
Bentabal	Sabrine

Table des Matières

- 1. Introduction et Contexte
 - 2. Architecture Technique
 - 3. TP 1 — Chiffrement Classique et Cryptanalyse
 - 4. TP 2 — Cryptographie Symétrique Moderne
 - 5. TP 3 — Cryptographie Asymétrique
 - 6. TP 4 — Fonctions de Hachage
 - 7. TP 5 — Signatures Numériques
 - 8. TP 6 — Application Sécurisée (Chat WebSocket)
 - 9. Interface Web et Tableau de Bord
 - 10. Outils Transversaux
 - 11. Résultats et Mesures de Performance
 - 12. Conclusions et Perspectives
 - 13. Annexes
-

1. Introduction et Contexte

1.1 Objectifs du Projet

Ce projet constitue l'aboutissement pratique du module de Cryptographie Appliquée. L'objectif est de concevoir et d'implémenter, depuis zéro, une **plateforme éducative complète** couvrant l'ensemble des techniques cryptographiques modernes — des chiffrements classiques du XIXe siècle jusqu'aux protocoles d'échange de clés sur courbes elliptiques.

La plateforme se présente sous deux formes complémentaires :

- **Une interface en ligne de commande (CLI)** permettant d'appeler chaque algorithme directement depuis le terminal, avec contrôle total des paramètres.
- **Un tableau de bord web interactif (CryptoLab Dashboard)** accessible depuis le navigateur, fournissant des visualisations pédagogiques, des cas de test prédéfinis et des animations pour illustrer les propriétés cryptographiques.

1.2 Langage et Justification du Choix Technologique

Le projet est entièrement développé en **Go (Golang)**. Ce choix est motivé par :

- La **gestion mémoire sûre** et l'absence de vulnérabilités de type buffer overflow.
- Le **package standard crypto/** très complet (AES, RSA, ECDSA, SHA, etc.).
- Les **goroutines et channels** permettant la gestion concurrente des connexions WebSocket.
- La **compilation statique** produisant un binaire unique facilement déployable.
- La performance native comparable au C pour les opérations cryptographiques.

1.3 Périmètre Fonctionnel

Le projet couvre 6 travaux pratiques :

TP	Thème	Algorithmes clés
TP 1	Chiffrement Classique	César, Vigenère, Hill, OTP
TP 2	Symétrique Moderne	AES, DES, 3DES, RC4, MARS, Serpent, Twofish, RC6
TP 3	Asymétrique	RSA, ElGamal, ECC, Diffie-Hellman
TP 4	Hachage	MD5, SHA-256, SHA-512, HMAC
TP 5	Signatures	RSA-PSS, RSA-PKCS#1v15, ECDSA P-256, ECDSA P-384
TP 6	Application Sécurisée	WebSocket + AES-256-CBC + HMAC-SHA256

2. Architecture Technique

2.1 Structure du Projet

```
crypto/  
├── cmd/  
│   └── main.go           # Point d'entrée CLI + serveur web  
├── internal/  
│   ├── AES/             # AES 128/192/256 bits, modes ECB/CBC  
│   ├── Affine/          # Chiffrement affine  
│   ├── Bluetooth/       # Communications Bluetooth sécurisées  
│   ├── Caesar/          # César + attaques (brute force, IC, dict.)  
│   ├── Chat/            # Serveur WebSocket, AES-CBC, HMAC  
│   ├── DES/             # DES avec S-boxes, Feistel, modes ECB/CBC  
│   ├── DiffieHellman/   # DH classique + MITM simulation  
│   ├── ECC/             # Arithmétique sur courbes elliptiques + ECDH  
│   ├── ElGamal/         # ElGamal + malléabilité + homomorphisme  
│   └── Election/        # Vote électronique (cryptosystème Paillier)
```

```

├── Hill/                # Chiffrement de Hill + attaque à clair connu
├── HMAC/                # HMAC-SHA256 / HMAC-SHA512
├── MARS/                # Algorithme MARS (finaliste AES)
├── MD5/                 # MD5 complet
├── OTP/                 # One-Time Pad + crib dragging
├── Playfair/            # Chiffrement de Playfair
├── RC4/                 # RC4 + attaque WEP + test de biais
├── RC6/                 # RC6 (finaliste AES)
├── RSA/                 # RSA multi-tailles + hybride OAEP
├── Serpent/             # Serpent (finaliste AES)
├── SHA256/              # SHA-256
├── SHA512/              # SHA-512
├── Signature/           # RSA-PSS, RSA-PKCS1v15, ECDSA, ElGamal sig.
├── Socket/              # TCP sécurisé TLS
├── TripleDES/           # 3DES clé 24 octets
├── Twofish/             # Twofish (finaliste AES)
├── Vigenere/            # Vigenère + test de Kasiski
├── analyzer/            # Analyse de fréquence, IC, avalanche
├── benchmark/           # Mesures de performance
├── core/                # Constantes et utilitaires partagés
├── modes/               # Modes de chiffrement (ECB, CBC, CTR)
├── web/
│   ├── handlers.go      # 1 557 lignes – logique HTTP et cryptographie
web
│   └── server.go        # Routage HTTP + WebSocket
└── templates/
    ├── base.html        # Layout commun (sidebar, header)
    ├── tp1.html         # Interface TP 1 (classique)
    ├── tp2.html         # Interface TP 2 (symétrique + image)
    ├── tp3.html         # Interface TP 3 (asymétrique)
    ├── tp4.html         # Interface TP 4 (hachage)
    ├── tp5.html         # Interface TP 5 (signatures)
    └── tp6.html         # Interface TP 6 (chat sécurisé)

```

2.2 Serveur Web et Routage

Le serveur HTTP est implémenté avec la bibliothèque standard `net/http`. Le routage définit **22 routes** distinctes :

```

// Extrait de internal/web/server.go
mux.HandleFunc("/tp1", TP1Handler)
mux.HandleFunc("/tp1/analyze", AnalyzeHandler)
mux.HandleFunc("/tp1/crib", CribHandler)
mux.HandleFunc("/tp2", TP2Handler)
mux.HandleFunc("/tp2/upload", ImageUploadHandler)
mux.HandleFunc("/tp2/benchmark", BenchmarkHandler)
mux.HandleFunc("/tp2/avalanche", AvalancheHandler)
mux.HandleFunc("/tp3/rsa-keygen", RSAKeyGenHandler)
mux.HandleFunc("/tp3/rsa-encrypt-text", RSATextEncryptHandler)
mux.HandleFunc("/tp3/rsa-decrypt-text", RSATextDecryptHandler)
mux.HandleFunc("/tp3/dh", DHHandler)

```

```
mux.HandleFunc("/tp3/elgamal-encrypt", ElGamalEncryptHandler)
mux.HandleFunc("/tp3/elgamal-forge", ElGamalForgeHandler)
mux.HandleFunc("/tp3/ecc-point", ECCPointHandler)
mux.HandleFunc("/tp3/ecdh", ECDHHandler)
mux.HandleFunc("/tp4/hash", HashHandler)
mux.HandleFunc("/tp5/signature", TP5SignatureHandler)
mux.HandleFunc("/tp6/chat", ChatHandler)
mux.HandleFunc("/ws/chat", chat.HandleWebSocketConnection)
mux.HandleFunc("/ws/rooms", RoomsHandler)
```

Le démarrage du serveur se fait via :

```
go run ./cmd/main.go -serve -port 8080
# Accès : http://localhost:8080
```

2.3 Templating Go

Le rendu HTML utilise le moteur `html/template` de Go avec un template de base (`base.html`) qui injecte le contenu de chaque TP dynamiquement selon le champ `ActiveTab` de la structure `PageData` :

```
type PageData struct {
    Title           string
    ActiveTab       string
    ActiveSubTab    string
    Result          *analyzer.AnalysisResult
    DHResult        map[string]interface{}
    RSAKeys         *RSAKeyPair
    RSATextResult   *RSATextResult
    ElGamalResult   *ElGamalEncryptResult
    ECCResult       *ECCResult
    ECDHResult      *ECDHResult
    Hashes          map[string]string
    HashAval        float64
    SignatureResult *SignatureResult
    Benchmarks      map[string]float64
    // ... et 15 autres champs
}
```

3. TP 1 — Chiffrement Classique et Cryptanalyse

3.1 Vue d'ensemble

Ce TP implémente les quatre grands chiffrements classiques ainsi que leurs attaques associées. L'interface web permet de chiffrer/déchiffrer des textes et d'observer en temps réel l'analyse de fréquence des lettres, comparée aux fréquences théoriques de l'anglais.

3.2 Chiffrement de César

Principe : Substitution monoalphabétique — chaque lettre est décalée de k positions dans l'alphabet.

$$E(x) = (x + k) \bmod 26$$
$$D(y) = (y - k) \bmod 26$$

Fichiers : `internal/Caesar/algo.go`, `internal/Caesar/attack.go`

Implémentation Go :

```
type CaesarAlgo struct {
    decalage int
    text     string
}

func (c *CaesarAlgo) Encrypt(word string) string { ... }
func (c *CaesarAlgo) Decrypt(word string) string { ... }
func (c *CaesarAlgo) BruteForce() []string { ... }
func (c *CaesarAlgo) CrackWithIC() int { ... }
func (c *CaesarAlgo) CrackWithDictionary() string { ... }
```

Attaques disponibles :

- **Brute force** : teste les 26 décalages possibles.
- **Indice de Coïncidence (IC)** : mesure la probabilité que deux lettres choisies au hasard soient identiques. Pour l'anglais, $IC \approx 0.065$; pour un texte uniforme, $IC \approx 0.038$.
- **Attaque par dictionnaire** : valide chaque décalage en vérifiant la présence de mots communs.

Interface web : champ de texte, slider pour le décalage (1–25), graphique Chart.js des fréquences lettres, affichage de l'IC calculé.

3.3 Chiffrement de Vigenère

Principe : Substitution polyalphabétique — chaque lettre est chiffrée avec un décalage différent selon la position dans la clé.

$$E(x_i) = (x_i + k_i) \bmod 26 \quad \text{où } k_i = \text{clé}[i \bmod \text{len}(\text{clé})]$$

Fichiers : `internal/Vigenere/algo.go`

Fonctionnalités :

- Chiffrement et déchiffrement avec clé alphabétique.
- **Test de Kasiski** : identifie les répétitions dans le texte chiffré pour estimer la longueur de la clé.

```
type KasiskiResult struct {
    Sequences      map[string][]int
    PossibleLengths []int
}
```

Interface web : champ clé alphanumérique, résultats du test de Kasiski avec les longueurs probables listées.

3.4 Chiffrement de Hill

Principe : Chiffrement matriciel — le texte est découpé en blocs de taille n et multiplié par une matrice clé K de dimension $n \times n$ modulo 26.

$$C = K \cdot P \pmod{26}$$

$$P = K^{-1} \cdot C \pmod{26}$$

Fichiers : [internal/Hill/algo.go](#), [internal/Hill/attack.go](#)

Implémentation :

```
type HillAlgo struct {
    matrix      [][]int
    matrixSize  int
}
```

Fonctionnalités :

- Support des matrices 2×2 et 3×3 .
- Calcul automatique de l'inverse modulaire de la matrice clé.
- **Attaque à clair connu (Known-Plaintext Attack)** : récupère la matrice clé à partir de paires clair/chiffré.

3.5 One-Time Pad (OTP)

Principe : Chiffrement par XOR avec une clé aléatoire de même longueur que le message. Prouvé théoriquement parfait par Shannon (1949).

$$C = P \oplus K$$

$$P = C \oplus K$$

Fichiers : [internal/OTP/algo.go](#), [internal/OTP/attack.go](#)

Vulnérabilité — Réutilisation de clé :

Si deux messages **P1** et **P2** sont chiffrés avec la même clé **K** :

$$C1 \oplus C2 = P1 \oplus P2$$

Cette propriété permet le **crib dragging** : en supposant un mot probable (**crib**) à une position, on extrait des fragments des deux messages.

Interface web :

- Formulaire de chiffrement/déchiffrement avec clé hexadécimale.
- Section "Attaque par réutilisation de clé" : entrée de C1 et C2, affichage du XOR résultant.

3.6 Analyse de Fréquence (Transversale à TP1)

Le module `internal/analyzer/` fournit :

- Comptage des fréquences de chaque lettre.
- Calcul de l'**Indice de Coïncidence** (IC).
- Comparaison avec les fréquences théoriques anglaises.
- Test de Kasiski pour la longueur de clé Vigenère.
- Calcul de l'**effet d'avalanche** entre deux hachés.

4. TP 2 — Cryptographie Symétrique Moderne

4.1 RC4 — Chiffrement par Flot

Principe : Chiffrement à flot basé sur une permutation pseudo-aléatoire de 256 octets.

Phases :

1. **KSA (Key Scheduling Algorithm)** : initialise le tableau d'état S[0..255].
2. **PRGA (Pseudo-Random Generation Algorithm)** : génère le keystream.

```
type RC4Algo struct {  
    state [256]byte  
}  
  
func (r *RC4Algo) InitRC4(key string)  
func (r *RC4Algo) Encrypt(plaintext string) string  
func (r *RC4Algo) WEPAttack() string  
func (r *RC4Algo) RC4BiasTest() map[byte]int
```

Attaque WEP : Le protocole WEP concatène un IV de 3 octets avec la clé. Cette faiblesse permet de reconstituer la clé avec seulement ~40 000 paquets capturés (attaque de Fluhrer, Mantin, Shamir).

4.2 DES — Data Encryption Standard

Principe : Chiffrement par blocs de 64 bits avec une structure de **réseau de Feistel** à 16 tours et une clé de 56 bits effectifs.

Fichiers : [internal/DES/algo.go](#)

Composants implémentés :

- Tables de permutation IP et IP^{-1} .
- Expansion E (32 $\rightarrow$ 48 bits).
- 8 S-boxes (substitution non-linéaire, cœur de la sécurité DES).
- Permutation P.
- Schedule de clé : génération des 16 sous-clés de 48 bits.

Modes de chiffrement :

Mode	Description	Propriété
ECB	Chaque bloc indépendamment	Déterministe, révèle les motifs
CBC	Bloc XOR-é avec le précédent	Non-déterministe avec IV

Visualisation par image :

```
func (d *DESAlgo) EncryptImage(imgData []byte) []byte // mode ECB
func (d *DESAlgo) CBCEncryptImage(imgData []byte) []byte // mode CBC
```

L'interface permet d'uploader une image PNG et de visualiser côte à côte :

- L'image originale
- La version chiffrée en mode **ECB** (les motifs visuels restent reconnaissables)
- La version chiffrée en mode **CBC** (visuellement aléatoire)

Ceci illustre de manière frappante pourquoi ECB est un mode non recommandé.

4.3 Triple DES (3DES)

Principe : Applique DES trois fois avec des clés distinctes : $C = E(K3, D(K2, E(K1, P)))$.

Fichiers : [internal/TripleDES/algo.go](#)

Clé de 24 octets (168 bits effectifs). La sécurité pratique est de ~ 112 bits (attaque meet-in-the-middle).

4.4 AES — Advanced Encryption Standard

Principe : Chiffrement par blocs de 128 bits avec clés 128, 192 ou 256 bits. Vainqueur du concours NIST en 2001.

Fichiers : [internal/AES/algo.go](#)

Implémentation complète :


```

func (a *AESAlgo) SubBytes(state [][]byte) [][]byte // S-Box
func (a *AESAlgo) ShiftRows(state [][]byte) [][]byte // Décalage cyclique
func (a *AESAlgo) MixColumns(state [][]byte) [][]byte // Multiplication
GF(2^8)
func (a *AESAlgo) AddRoundKey(state, key [][]byte) [][]byte
func (a *AESAlgo) KeyExpansion() [][][]byte
func (a *AESAlgo) EncryptECB(data []byte) []byte
func (a *AESAlgo) EncryptCBC(data []byte, iv []byte) []byte

```

Nombre de tours :

- AES-128 : 10 tours
- AES-192 : 12 tours
- AES-256 : 14 tours

4.5 Les Finalistes du Concours AES (NIST)

En plus d'AES, le projet implémente les quatre autres finalistes du concours NIST (1997–2001) :

Algorithme	Structure	Tours	Particularité
MARS	Feistel hétérogène de Type 3	20	S-box dépendante des données
Serpent	Réseau SPN	32	Marge de sécurité maximale
Twofish	Feistel modifiée	16	S-box dépendant de la clé
RC6	Feistel avec rotations dépendantes	20	Proposé par RSA Security

Fichiers : [internal/MARS/](#), [internal/Serpent/](#), [internal/Twofish/](#), [internal/RC6/](#)

4.6 Benchmark de Performance

Le handler [/tp2/benchmark](#) mesure le temps de chiffrement d'un payload de **1 Mo** pour chaque algorithme :

```

func BenchmarkHandler(w http.ResponseWriter, r *http.Request) {
    payload := make([]byte, 1024*1024)
    key := "1234567890123456"
    aesAlgo, _ := internalaes.InitAES(key)
    start := time.Now()
    aesAlgo.EncryptECB(payload)
    benchmarks["AES"] = float64(time.Since(start).Milliseconds())
    // ...
}

```

4.7 Effet d'Avalanche sur les Chiffrements Symétriques

Le handler `/tp2/avalanche` modifie **un seul bit** de l'entrée et mesure le pourcentage de bits différents dans la sortie. Un bon chiffrement produit environ 50% de bits changés (propriété d'avalanche).

```
data2[0] ^= 1 // Flip du premier bit
avalanche := analyzer.CalculateAvalanche(enc1, enc2)
```

5. TP 3 — Cryptographie Asymétrique

5.1 Diffie-Hellman (DH)

Principe : Protocole d'échange de clés sans transmission de secret, basé sur la difficulté du **logarithme discret** dans un groupe cyclique fini.

```
Alice choisit a (privé) → A = ga mod p (public)
Bob choisit b (privé) → B = gb mod p (public)
Secret partagé : S = Ba mod p = Ab mod p = ga·b mod p
```

Fichiers : [internal/DiffieHellman/algo.go](#), [internal/DiffieHellman/attack.go](#)

Simulation MITM (Man-in-the-Middle) :

Mallory s'interpose entre Alice et Bob, créant deux sessions DH distinctes :

- Alice–Mallory : secret $S_1 = g^{(am)} \bmod p$
- Mallory–Bob : secret $S_2 = g^{(bm)} \bmod p$

Mallory déchiffre avec S_1 , relit le message, re-chiffre avec S_2 .

Contre-mesure simulée : authentification ECDSA des clés publiques échangées — si Alice signe sa clé publique, Mallory ne peut pas la falsifier.

Interface web :

- Colonne Alice / Canal (ou Mallory) / Bob.
- Bouton "Enable MITM" pour basculer en mode attaque.
- Bouton "Enable Signatures" pour activer la contre-mesure.
- Bouton "Use 512-bit Prime" pour générer un premier cryptographique fort.

```
// Génération d'un premier de 512 bits avec rand.Prime
p, _ := rand.Prime(rand.Reader, 512)
```

5.2 RSA — Rivest-Shamir-Adleman

Principe : Basé sur la difficulté de factoriser le produit de deux grands nombres premiers.

```

n = p × q                (module RSA)
e tel que gcd(e, φ(n)) = 1 (exposant public)
d = e-1 mod φ(n)        (clé privée)
C = Me mod n            (chiffrement)
M = Cd mod n            (déchiffrement)

```

Fichiers : `internal/RSA/algo.go`, `internal/RSA/hybrid.go`

```

type RSAAlgo struct {
    n *big.Int
    e *big.Int
    d *big.Int
}

```

Tailles de clés supportées : 512, 1024, 2048 bits.

Génération de clés PEM :

```

privDER, _ := x509.MarshalPKCS8PrivateKey(privateKey)
privPEM := pem.EncodeToMemory(&pem.Block{Type: "PRIVATE KEY", Bytes:
privDER})
pubDER, _ := x509.MarshalPKIXPublicKey(&privateKey.PublicKey)
pubPEM := pem.EncodeToMemory(&pem.Block{Type: "PUBLIC KEY", Bytes: pubDER})

```

5.2.1 Chiffrement Hybride RSA + AES (OAEP)

RSA seul ne peut chiffrer que de petits messages (limité par la taille de `n`). Le **chiffrement hybride** résout ce problème :

Étape 1 — Chiffrement (handler `/tp3/rsa-encrypt-text`) :

1. Générer une clé AES-256 aléatoire (`sessionKey`, 32 octets).
2. Chiffrer le message avec AES-256-CBC (IV aléatoire + PKCS#7 padding).
3. Chiffrer `sessionKey` avec RSA-OAEP (SHA-256).

```

// Génération de la session key
sessionKey := make([]byte, 32)
rand.Read(sessionKey)

// Chiffrement AES-CBC
iv := make([]byte, aes.BlockSize)
rand.Read(iv)
paddedText := pkcs7Pad([]byte(plaintext), aes.BlockSize)
cipher.NewCBCEncrypter(aesBlock, iv).CryptBlocks(ciphertext, paddedText)

```

```
// Encapsulation RSA-OAEP
wrappedKey, _ := rsa.EncryptOAEP(sha256.New(), rand.Reader, rsaPubKey,
sessionKey, nil)
```

Étape 2 — Déchiffrement (handler /tp3/rsa-decrypt-text) :

1. Décrypter `wrappedKey` avec la clé privée RSA-OAEP → récupère `sessionKey`.
2. Décrypter le ciphertext AES-CBC → récupère le message original.

L'interface web est un assistant 3 étapes :

- Étape 1 : Générer les clés → affiche PEM public + PEM privé.
- Étape 2 : Chiffrer → auto-remplit le champ "Clé Publique" depuis l'étape 1.
- Étape 3 : Déchiffrer → auto-remplit le champ "Clé Privée" et le champ "Ciphertext" depuis l'étape 2.

Des boutons JavaScript `loadKeysToForms()` et `loadCiphertextToDecrypt()` assurent le flux de données entre étapes sans copier-coller manuel.

5.3 ElGamal

Principe : Chiffrement asymétrique basé sur le problème du logarithme discret, proposé par Taher ElGamal en 1985.

```
type ElGamalAlgo struct {
    p          *big.Int
    g          *big.Int
    PrivateKey *big.Int
    PublicKey  *big.Int
}
```

Propriétés démontrées dans l'interface :

Non-déterminisme : Chiffrer deux fois le même message `m` donne deux ciphertexts différents (car `k` est aléatoire à chaque fois).

```
Encryption 1 : (C1, C2) = (gk mod p, m · Yk mod p)
Encryption 2 : (C1', C2') ≠ (C1, C2) [car k' ≠ k]
```

Malléabilité (attaque) : Sans intégrité, un attaquant peut modifier le ciphertext pour multiplier le message par n'importe quel facteur :

```
(C1, C2 · t mod p) → déchiffrement donne m · t
```

L'interface affiche la valeur originale et la valeur forgée, illustrant pourquoi ElGamal pur (sans MAC) ne doit pas être utilisé en production.

5.4 Cryptographie sur Courbes Elliptiques (ECC)

Principe : Les opérations cryptographiques sont réalisées sur des points d'une courbe elliptique définie sur un corps fini. La sécurité repose sur le **problème du logarithme discret elliptique (ECDLP)**.

Courbe : $y^2 = x^3 + ax + b \pmod{p}$

Fichiers : `internal/ECC/algo.go`

```
type Point struct {
    X *big.Int
    Y *big.Int
}

type Curve struct {
    A, B, P *big.Int
    Name    string
}
```

Arithmétique implémentée :

- Addition de points : $P + Q$
- Doublement : $P + P$
- Multiplication scalaire : $k \cdot P$ (double-and-add)

Interface web — Multiplication scalaire :

La page affiche les étapes intermédiaires de la multiplication $k \cdot G$ sur la courbe $y^2 = x^3 + 7 \pmod{97}$ (simplifiée pour l'illustration). Des boutons prédéfinis $k=7$ et $k=13$ permettent de visualiser les étapes rapidement.

```
func calculateECCMult(px, py, k int) *ECCResult {
    p := 97 ; a := 0 ; b := 7
    for i := 1; i < k; i++ {
        logs = append(logs, fmt.Sprintf("Step %d: (%d,%d)", i+1, x, y))
        x, y = pointAdd(x, y, px, py, p, a, b)
    }
}
```

ECDH — Elliptic Curve Diffie-Hellman :

Échange de clés utilisant la courbe **P-256** (NIST) :

```

alicePriv, _ := ecdsa.GenerateKey(elliptic.P256(), rand.Reader)
bobPriv, _ := ecdsa.GenerateKey(elliptic.P256(), rand.Reader)

// Secret partagé = Alice · (Bob_pubkey) = Bob · (Alice_pubkey)
sharedX, _ := elliptic.P256().ScalarMult(
    bobPriv.PublicKey.X, bobPriv.PublicKey.Y,
    alicePriv.D.Bytes(),
)
// Dérivation AES-256 :
sharedSecret := sha256.Sum256(append(sharedX.Bytes(), sharedY.Bytes()...))

```

Avantage de l'ECC sur RSA : Une clé ECDSA de 256 bits offre une sécurité équivalente à une clé RSA de 3072 bits.

6. TP 4 — Fonctions de Hachage

6.1 Vue d'ensemble

Ce TP se concentre exclusivement sur les fonctions de hachage cryptographiques et leurs propriétés fondamentales. L'interface affiche un **Hash Inspector** comparant MD5, SHA-256 et SHA-512 sur le même message, suivi d'une visualisation de l'effet d'avalanche.

6.2 MD5

Sortie : 128 bits (32 caractères hexadécimaux)

Structure : 4 tours de 16 opérations chacun, fonctions booléennes F, G, H, I.

```

hMD5 := md5.Sum([]byte(inputText))
hashes["MD5"] = fmt.Sprintf("%x", hMD5)

```

Statut de sécurité : MD5 est **cassé** depuis 2004 (Wang et Yu). Des collisions peuvent être trouvées en quelques secondes. Utilisé uniquement pour les sommes de contrôle non-sécuritaires.

6.3 SHA-256

Sortie : 256 bits (64 caractères hexadécimaux)

Structure : 64 tours de compression avec 8 variables de travail (a, b, c, d, e, f, g, h) et 64 constantes K_i dérivées des racines cubiques de nombres premiers.

```

h256 := sha256.Sum256([]byte(inputText))
hashes["SHA-256"] = fmt.Sprintf("%x", h256)

```

Standard actuel : SHA-256 est le standard de facto pour les certificats TLS, Bitcoin, Git, les signatures numériques.

6.4 SHA-512

Sortie : 512 bits (128 caractères hexadécimaux)

Structure : 80 tours, mots de 64 bits (optimal sur architectures 64 bits).

```
h512 := sha512.Sum512([]byte(inputText))
hashes["SHA-512"] = fmt.Sprintf("%x", h512)
```

6.5 HMAC — Keyed-Hash Message Authentication Code

Principe : Combine une fonction de hachage avec une clé secrète pour garantir à la fois l'**intégrité** et l'**authenticité** des données.

$$\text{HMAC}(K, m) = H((K \oplus \text{opad}) \parallel H((K \oplus \text{ipad}) \parallel m))$$

Fichiers : `internal/HMAC/algo.go`

```
func (h *HMACAlgo) Sign(message string) string
func (h *HMACAlgo) Verify(message, tag string) bool
```

6.6 Effet d'Avalanche des Fonctions de Hachage

La propriété d'avalanche exige qu'un changement d'**un seul bit** dans l'entrée provoque un changement d'environ **50%** des bits dans la sortie.

Démonstration interactive :

- Boutons prédéfinis : "Hello World", "hello world", "password123", "The quick brown fox...", "(vide)"
- Modification automatique du premier bit.
- Affichage du pourcentage de bits différents entre $H(m)$ et $H(m')$.

```
data2[0] ^= 1 // flip du premier bit
hA1 := sha256.Sum256(data1)
hA2 := sha256.Sum256(data2)
avalanche := analyzer.CalculateAvalanche(hA1[:], hA2[:])
```

7. TP 5 — Signatures Numériques

7.1 Principe des Signatures Numériques

Une signature numérique permet de garantir :

1. **Authenticité** — seul le détenteur de la clé privée peut signer.

2. **Intégrité** — toute modification du message invalide la signature.
3. **Non-répudiation** — le signataire ne peut pas nier avoir signé.

Schéma général :

```
Signature :  $\sigma = \text{Sign}(\text{privKey}, H(m))$   
Vérification :  $\text{Verify}(\text{pubKey}, m, \sigma) \rightarrow \{\text{vrai}, \text{faux}\}$ 
```

7.2 RSA-PSS (Probabilistic Signature Scheme)

Fichiers : [internal/Signature/RSA.go](#)

RSA-PSS est le mode de signature RSA recommandé par PKCS#1 v2. Il ajoute un **sel aléatoire** avant le hachage, rendant deux signatures du même message différentes.

```
sig, _ := signature.InitRSASignature(2048)  
sigHex, _ := sig.SignPSS(message) // Signature  
verified := sig.VerifyPSS(message, sigHex) // Vérification  
pubKey := sig.GetPublicKey()
```

Tailles de clés : 1024, 2048, 4096 bits.

7.3 RSA-PKCS#1 v1.5

Mode de signature RSA plus ancien mais encore largement déployé (TLS 1.2, PGP historique).
Déterministe — la même clé sur le même message produit toujours la même signature. Vulnérable à l'**attaque de Bleichenbacher** (padding oracle) si mal implémenté côté déchiffrement.

```
sigHex, _ := sig.SignPKCS1v15(message)  
verified := sig.VerifyPKCS1v15(message, sigHex)
```

7.4 ECDSA — Elliptic Curve Digital Signature Algorithm

Fichiers : [internal/Signature/ECDSA.go](#)

ECDSA utilise l'arithmétique sur courbes elliptiques. Pour chaque signature, un entier **k** aléatoire est généré.

```

$$r = (k \cdot G).x \bmod n$$

$$s = k^{-1} \cdot (H(m) + r \cdot \text{privKey}) \bmod n$$

```

Courbes supportées : P-256, P-384 (NIST), P-224, P-521.


```
sig, _ := signature.InitECDSA("P-256")
sigHex, _ := sig.Sign(message)
verified := sig.Verify(message, sigHex)
pubKey := sig.GetPublicKey()
```

Avantage clé : Signatures beaucoup plus courtes qu'en RSA — une signature ECDSA P-256 fait 64 octets contre 256 octets pour RSA-2048.

Vulnérabilité — Réutilisation de k :

Si k est réutilisé pour deux messages différents m_1 et m_2 :

$$s_1 - s_2 = k^{-1}(H(m_1) - H(m_2)) \bmod n$$

$$\rightarrow k = (H(m_1) - H(m_2)) \cdot (s_1 - s_2)^{-1} \bmod n$$

$$\rightarrow \text{privKey} = (s \cdot k - H(m)) \cdot r^{-1} \bmod n$$

La clé privée est entièrement compromise. Sony PlayStation 3 a été victime de cette attaque en 2010.

7.5 Interface Web — Flux Sign → Verify

L'interface TP5 est organisée en deux étapes enchaînées :

Étape 1 (Signer) :

- Sélecteur d'algorithme (RSA-PSS, RSA-PKCS1v15, ECDSA P-256, ECDSA P-384).
- Boutons de messages prédéfinis : "Hello World", "Virement bancaire 5000€", "Contrat signé le 17/05/2026", "Je vote pour le candidat A".
- Affichage : hexadécimal de la signature + clé publique (PEM tronqué).
- Bouton **"Copier → Formulaire de Vérification"** (JavaScript).

Étape 2 (Vérifier) :

- Champs auto-remplis depuis l'étape 1.
- Bouton "Vérifier" → affiche ✓ **Signature valide** ou ✗ **Invalide**.
- Bouton **"Falsifier + Vérifier"** → ajoute un espace au message avant vérification → toujours invalide, illustrant l'intégrité.

```
function tamperAndVerify() {
    document.getElementById('verify_message').value += ' ';
    document.querySelector('#step2 form').submit();
}
```

8. TP 6 — Application Sécurisée (Chat WebSocket)

8.1 Architecture du Chat Sécurisé

Ce TP implémente une application de **messagerie instantanée chiffrée** utilisant le protocole WebSocket, avec des salons nommés, chiffrement AES-256-CBC par connexion et HMAC-SHA256 pour l'intégrité.

Fichiers : `internal/Chat/websocket.go` (353 lignes)

```
Client WebSocket → ws://localhost:8080/ws/chat?room=R&username=U
                  |
                  [Upgrade HTTP → WebSocket]
                  |
                  ChatServer.HandleWebSocket()
                  |
                  Room.broadcast → Tous les clients du salon
```

8.2 Structures de Données

```
type Client struct {
    conn    *websocket.Conn
    send    chan []byte    // Canal d'envoi
    username string
    roomID  string
    key     []byte        // Clé AES-256 par client (32 octets)
}

type Room struct {
    roomID    string
    clients   map[*Client]bool
    broadcast chan []byte    // Messages à diffuser
    register  chan *Client
    unregister chan *Client
    mutex     sync.RWMutex
}

type ChatServer struct {
    rooms      map[string]*Room
    roomsMutex sync.RWMutex
}
```

8.3 Chiffrement des Messages

Chaque client reçoit une **clé AES-256 aléatoire** à la connexion. Les messages sont chiffrés avant transmission :

```
func (c *Client) encryptMessage(plaintext string) (string, string) {
    block, _ := aes.NewCipher(c.key) // AES-256
    iv := make([]byte, aes.BlockSize)
    rand.Read(iv)                    // IV aléatoire
```

```

    padded := wsPKCS7Pad([]byte(plaintext)) // PKCS#7
    ciphertext := make([]byte, len(padded))
    cipher.NewCBCEncrypter(block, iv).CryptBlocks(ciphertext, padded)

    encrypted := hex.EncodeToString(iv) + ":" +
hex.EncodeToString(ciphertext)
    mac := computeHMAC(plaintext, c.key) // HMAC-SHA256

    return encrypted, mac
}

```

Correction du bug critique : La version initiale ne paddait pas le plaintext avant `CryptBlocks`, causant un **panic** `"crypto/cipher: input not full blocks"` à l'exécution. Le correctif implémente PKCS#7 :

```

func wsPKCS7Pad(data []byte) []byte {
    padding := aes.BlockSize - len(data)%aes.BlockSize
    pad := make([]byte, padding)
    for i := range pad { pad[i] = byte(padding) }
    return append(data, pad...)
}

func wsPKCS7Unpad(data []byte) []byte {
    if len(data) == 0 { return data }
    p := int(data[len(data)-1])
    if p == 0 || p > aes.BlockSize || p > len(data) { return data }
    return data[:len(data)-p]
}

```

8.4 Gestion Multi-Salons Concurrente

Chaque salon tourne sa propre goroutine de dispatch :

```

go func() {
    for {
        select {
        case client := <-room.register:
            room.clients[client] = true
        case client := <-room.unregister:
            delete(room.clients, client)
            close(client.send)
        case message := <-room.broadcast:
            for client := range room.clients {
                client.send <- message // Non-bloquant via select
            }
        }
    }
}

```

```
}
}()
```

Le server-sent **keepalive** envoie un ping WebSocket toutes les 30 secondes, et un read deadline de 60 secondes détecte les clients déconnectés.

8.5 Interface Web — Wire Inspector

L'interface TP6 comporte plusieurs panneaux :

Panneau de connexion :

- Champ nom d'utilisateur + nom du salon.
- Boutons prédéfinis : `#crypto-class`, `#tp6-demo`, `#secure-chat`.
- Connexion WebSocket via JavaScript.

Panneau de chat :

- Messages envoyés (bulles vertes, alignés à droite).
- Messages reçus (bulles grises, alignés à gauche).
- Timestamp de chaque message.

Wire Inspector (panneau éducatif) :

Ce panneau utilise l'**API Web Crypto du navigateur** (`window.crypto.subtle`) pour chiffrer le dernier message côté client et afficher les octets réels sur le réseau :

```
async function showInspector(message) {
  const key = await generateAESKey();           // AES-256
  const { iv, ciphertext } = await aesEncrypt(key, message);
  const hmac = await computeHMAC(key, message);

  document.getElementById('iv-hex').textContent = toHex(iv);
  document.getElementById('cipher-hex').textContent = toHex(ciphertext);
  document.getElementById('hmac-hex').textContent = toHex(hmac);
}
```

Cela permet à l'étudiant de voir concrètement ce qui circule sur le réseau.

Panneau de vote homomorphique :

Simulation d'un vote électronique chiffré. Chaque vote est "chiffré" côté client et le décompte affiché illustre la propriété d'homomorphisme :

$$\text{Enc}(\text{vote1}) + \text{Enc}(\text{vote2}) = \text{Enc}(\text{vote1} + \text{vote2})$$

9. Interface Web et Tableau de Bord

9.1 Design System

L'interface utilise un thème cybersécurité sombre ("cyber dark") :

```
/* Palette de couleurs */
background: #0d1117 /* fond global */
panel:      #161b22 /* panneaux */
border:     #30363d /* séparateurs */
accent:     #22c55e /* vert terminal */
text:       #d1d5db /* gris clair */
font:       'Fira Code', monospace /* police développeur */
```

Framework CSS : **Tailwind CSS** (CDN, utilitaire).
Graphiques : **Chart.js** (fréquences de lettres, benchmarks).

9.2 Sidebar Navigation

La sidebar gauche est commune à toutes les pages via `base.html`. Elle met en surbrillance l'onglet actif :

```
<a href="/tp5" class="flex items-center p-3 rounded-lg
  {{if eq .ActiveTab "tp5"}}
    bg-green-900/20 text-green-400 border border-green-500/50
  {{else}}
    hover:bg-gray-800
  {{end}}">
  📄 TP 5: Signatures
</a>
```

9.3 Interactivité JavaScript

Chaque TP intègre des fonctions JavaScript pour améliorer l'expérience pédagogique :

TP	Fonctions clés
TP1	Graphique fréquences lettres (Chart.js)
TP2	Affichage comparatif ECB/CBC, jauge d'avalanche
TP3	<code>loadKeysToForms()</code> , <code>loadCiphertextToDecrypt()</code> , <code>fillForgeFromEncrypt()</code> , <code>setK(val)</code>
TP4	Boutons prédéfinis ("Hello World", "password123", etc.), comparateur de hachés
TP5	<code>copyToVerify()</code> , <code>tamperAndVerify()</code> , <code>setMsg(text)</code>
TP6	WebSocket JS natif, <code>showInspector()</code> , <code>castVote()</code> , <code>resetVotes()</code> , <code>aesEncrypt()</code> (Web Crypto API)

10. Outils Transversaux

10.1 Module d'Analyse ([internal/analyzer/](#))

```
func Analyze(text string) AnalysisResult
func KasiskiExamination(text string, minLen int) KasiskiResult
func CalculateAvalanche(data1, data2 []byte) float64
```

AnalysisResult contient :

- Fréquences de chaque lettre.
- Indice de coïncidence calculé.
- Texte analysé + longueur.
- Fréquences de référence anglaise pour comparaison.

10.2 Module de Benchmark ([internal/benchmark/](#))

Mesure les performances de :

- Hachage : MD5, SHA-256, SHA-512 (débit en Mo/s).
- Chiffrement symétrique : DES, 3DES, AES (ms/Mo).
- Asymétrique : génération de clé RSA, opérations ECDSA.

10.3 Chiffrement de Playfair et Affine

Playfair ([internal/Playfair/](#)) : chiffrement par bigrammes sur une grille 5×5.

Affine ([internal/Affine/](#)) : $E(x) = (a \cdot x + b) \bmod 26$ avec $\gcd(a, 26) = 1$.

11. Résultats et Mesures de Performance

11.1 Benchmarks de Chiffrement Symétrique

Mesures effectuées sur un payload de 1 Mo avec Go standard library :

Algorithme	Temps (ms)	Débit approximatif
AES-128 ECB	~3–5 ms	~200–300 Mo/s
AES-256 CBC	~4–6 ms	~160–250 Mo/s
DES ECB	~15–25 ms	~40–70 Mo/s
3DES CBC	~45–60 ms	~16–22 Mo/s

11.2 Benchmarks de Chiffrement Hybride

Opération	Temps
Chiffrement hybride (AES+RSA key wrap) 1 Mo	< 10 ms

Opération	Temps
RSA-2048 pur sur 1 Mo	> 1 000 ms (limité par taille de n)

Ce résultat illustre concrètement pourquoi le chiffrement hybride est indispensable pour les grands volumes de données.

11.3 Effet d'Avalanche — Valeurs typiques

Algorithme	Avalanche typique
AES-256 (flip 1 bit entrée)	49–51%
DES (flip 1 bit)	47–53%
SHA-256 (flip 1 bit)	48–52%
César (flip 1 bit)	~4% (très faible — pas d'avalanche)

11.4 Comparaison RSA vs ECDSA

Critère	RSA-2048	ECDSA P-256
Taille clé privée	2048 bits	256 bits
Taille signature	256 octets	64 octets
Niveau de sécurité	~112 bits	~128 bits
Génération de clé	~50–200 ms	< 1 ms
Signature	~5–10 ms	< 1 ms
Vérification	< 1 ms	< 1 ms

12. Conclusions et Perspectives

12.1 Bilan des Réalisations

Ce projet a permis de concevoir et d'implémenter une **plateforme cryptographique académique complète** comprenant :

- **4 chiffrements classiques** : César, Vigenère, Hill, OTP avec leurs attaques associées.
- **8 chiffrements symétriques modernes** : RC4, DES, 3DES, AES (128/192/256), MARS, Serpent, Twofish, RC6.
- **4 systèmes asymétriques** : RSA (hybride OAEP), ElGamal, ECC/ECDH, Diffie-Hellman (avec MITM).
- **4 fonctions de hachage** : MD5, SHA-256, SHA-512, HMAC.
- **4 schémas de signature** : RSA-PSS, RSA-PKCS#1v15, ECDSA P-256, ECDSA P-384.
- **1 application sécurisée** : chat WebSocket multi-salons, AES-256-CBC, HMAC-SHA256.
- **1 tableau de bord web** entièrement interactif avec 22 routes HTTP et 7 templates.

12.2 Compétences Développées

Domaine	Compétences acquises
Cryptographie	Compréhension mathématique des algorithmes, modes de chiffrement, attaques
Programmation	Go avancé : goroutines, channels, interfaces, <code>crypto/</code> <code>stdlib</code>
Sécurité	Identification et correction de vulnérabilités (padding oracle, réutilisation de clé)
Web	HTTP/WebSocket, templates Go, JavaScript natif, Web Crypto API
Pédagogie	Conception d'interfaces éducatives avec cas de test prédéfinis

12.3 Points Techniques Notables

1. **Bug PKCS#7 corrigé** : La version initiale du chat WebSocket provoquait un panic `"crypto/cipher: input not full blocks"`. Le correctif implémente le padding PKCS#7 correct.
2. **Chiffrement hybride cohérent** : Flux 3 étapes (Générer → Chiffrer → Déchiffrer) avec auto-remplissage JavaScript des champs, rendant le processus RSA+AES visuellement compréhensible.
3. **Wire Inspector TP6** : Utilisation de l'API Web Crypto du navigateur pour montrer les octets réels AES-CBC + IV + HMAC qui circulent sur le réseau — sans dépendances externes.
4. **Simulation MITM Diffie-Hellman** : Modélisation complète avec Alice, Mallory et Bob, affichant les deux secrets partagés distincts calculés par Mallory.

12.4 Perspectives d'Amélioration

- Implémentation du **cryptosystème de Paillier** complet (homomorphisme additif) pour le vote électronique.
- Ajout de **tests unitaires** avec `go test` pour chaque algorithme (vecteurs de test NIST).
- Support **TLS 1.3** dans le module Socket.
- Implémentation de **ChaCha20-Poly1305** (alternative moderne à AES-GCM).
- Ajout du **Zero-Knowledge Proof** (preuve de connaissance sans révélation).
- Déploiement **Docker** pour faciliter la démonstration.

13. Annexes

Annexe A — Commandes de Lancement

```
# Démarrer le tableau de bord web
go run ./cmd/main.go -serve -port 8080

# Compiler le projet entier
go build ./...

# Tester un algorithme en CLI
go run ./cmd/main.go -algo caesar -encrypt "HELLO" -shift 3
```



```
# Connexion WebSocket de test  
ws://localhost:8080/ws/chat?room=demo&username=Alice
```

Annexe B — Routes HTTP

Méthode	Route	Description
GET	/tp1	Interface chiffrement classique
POST	/tp1/analyze	Chiffrement/déchiffrement + analyse
POST	/tp1/crib	Attaque crib dragging OTP
GET	/tp2	Interface symétrique
POST	/tp2/upload	Upload image + chiffrement ECB/CBC
GET	/tp2/benchmark	Benchmark AES
POST	/tp2/avalanche	Calcul effet d'avalanche
GET	/tp3	Interface asymétrique
POST	/tp3/dh	Échange Diffie-Hellman (+ MITM)
GET	/tp3/dh-large	DH avec premier 512 bits
POST	/tp3/rsa-keygen	Génération paire de clés RSA PEM
POST	/tp3/rsa-encrypt-text	Chiffrement hybride RSA+AES
POST	/tp3/rsa-decrypt-text	Déchiffrement hybride
POST	/tp3/benchmark	Benchmark hybride vs RSA pur
POST	/tp3/elgamal-encrypt	Chiffrement ElGamal (×2)
POST	/tp3/elgamal-forge	Attaque malléabilité
POST	/tp3/ecc-point	Multiplication scalaire ECC
GET	/tp3/ecdh	Échange ECDH P-256
GET	/tp4	Interface hachage
POST	/tp4/hash	MD5 + SHA-256 + SHA-512 + avalanche
GET	/tp5	Interface signatures
POST	/tp5/signature	Sign/Verify (RSA-PSS, ECDSA, etc.)
GET	/tp6	Interface chat sécurisé
WS	/ws/chat	WebSocket chat AES-CBC
GET	/ws/rooms	Liste des salons actifs (JSON)

Annexe C — Bibliothèques et Dépendances

Bibliothèque	Usage
crypto/aes	AES standard library Go
crypto/cipher	Modes CBC, CTR
crypto/rsa	RSA OAEP, PSS, PKCS1v15
crypto/ecdsa	ECDSA P-256/P-384
crypto/elliptic	Courbes NIST
crypto/x509	Sérialisation PEM
encoding/pem	Encodage PEM
math/big	Arithmétique grands entiers
crypto/rand	Générateur aléatoire cryptographique
github.com/gorilla/websocket	Serveur WebSocket
html/template	Rendu HTML sécurisé
Tailwind CSS (CDN)	Framework CSS utilitaire
Chart.js (CDN)	Graphiques interactifs
Web Crypto API	Cryptographie côté navigateur

Date : Mai 2026

École : USTHB — Université des Sciences et de la Technologie Houari Boumédiène

Département : Ingénierie des Systèmes — Spécialité Cybersécurité